

1.2.4. Pattern - 2

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Sample Test Cases

numberP...

```
1 a=int(input())
2 for i in range(1,a+1):
3     for j in range(1,i+1):
4         print(j,end=" ")
5     print()
```

Average time0.005 s4.75 ms

Maximum time0.006 s6.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 15 ms

Debug

Test cases

Expected output	Actual output
5	5
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5

Terminal

1.2.3. Pattern - 1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases

rightangl...

```
1 a=int(input())
2 for i in range(1,a+1):
3     print("*" * i)
```

Average time0.004 s3.67 msMaximum time0.005 s5.00 ms

2 out of 2 shown test case(s) passed4 out of 4 hidden test case(s) passed

Test case 14 msDebug

Expected output

Actual output

55

* * *

* * * *

* * * * *

* * * * * *

* * * * * * *

Terminal

Test cases

1.2.2. Fibonacci Series

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

- A single integer n representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases

fibonacci....

```

1  def fibonacci(n):
2
3      # write the code..
4
5      return fibonacci(..)
6
7
8  n = int(input())
9  for i in range(1, n + 1):
10     print(fibonacci(i), end=" ")
11
12 def fibonacci(n):
13     if n==1:
14         return 0
15     elif n==2:
16         return 1
17     else:
18         return fibonacci(n-1) + fibonacci(n-2)
19
20

```

Average time: 0.009 s (9.50 ms) | Maximum time: 0.022 s (22.00 ms)

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 (5 ms) Debug

Expected output	Actual output
0 1 1 2 3	5

Test case 2 (4 ms)

Terminal Test cases

1.2.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses. The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n , the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

- If the student passes all courses:
- Print the aggregate percentage (formatted to two decimal places).
 - Print the grade based on the aggregate percentage.
- If the student fails any course (marks < 40 in any course), print:
- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

Sample Test Cases

```

1 n = int(input())
2 marks = list(map(int, input().split()))
3 if any(mark < 40 for mark in marks):
4     print("Fail")
5 else:
6     percentage = sum(marks)/n
7     if (percentage>75):
8         grade = "Distinction"
9     elif (percentage>=60):
10        grade = "First Division"
11    elif (percentage>=50):
12        grade = "Second Division"
13    else:
14        grade = "Third Division"
15
16    print(f"Aggregate Percentage: {percentage:.2f}")
17    print(f"Grade: {grade}")
    
```

Average time 0.007 s Maximum time 0.009 s
6.80 ms 9.00 ms

5 out of 5 shown test case(s) passed

5 out of 5 hidden test case(s) passed

Test Case 1 9 ms Debug

Expected output

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Actual output

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Terminal Test cases

< Prev Reset Submit Next >

1.1.5. Multiplication Table

00:43

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

Print the multiplication table for the given number in the format:

<number> x <multiplier> = <result>

Note:

The character "x" is used in the output to represent multiplication.

Refer to the visible test-cases for more clarity.

Sample Test Cases

multiplica...

```
1 # Read the integer input
2 n = int(input())
3
4 # Loop from 1 to 10 and print the multiplication table
5 for i in range(1, 10 + 1):
6     print(f"{n} x {i} = {n * i}")
7
8
9
```

Average time

0.004 s

4.25 ms

Maximum time

0.006 s

6.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

Expected output

8

8 x 1 = 8

8 x 2 = 16

8 x 3 = 24

8 x 4 = 32

8 x 5 = 40

Actual output

8

8 x 1 = 8

8 x 2 = 16

8 x 3 = 24

8 x 4 = 32

8 x 5 = 40

Terminal

Test cases

< Prev

Reset

Submit

Next >

CODETANTRA

[Home](#) [Learn Anywhere](#)

202501090075@mitaoe.ac.inSupportLogout

1.1.4. Reverse a Number

01.4s

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

The input is a single integer.

Output Format:

The output prints a single integer, which is the reversed number.

Sample Test Cases

reverseN...

```
1 #write your code here...
2 number = int(input())
3 reverse_number = int(str(number)[::-1])
4 print(reverse_number)
5
6
```

Average time0.008 s8.00 ms

Maximum time0.014 s14.00 ms

2 out of 2 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 114 ms

Expected output5367

Actual output5367

7635

Test case 29 ms

TerminalTest cases

< Prev

Reset

Submit

Next >

CODETANTRA

HomeLearn Anywhere

202501090075@mitaoe.ac.inSupportLogout

1.1.3. Days between Two Dates

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format *YYYY-MM-DD*.
- The second line contains the second date in the format *YYYY-MM-DD*.

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

Sample Test Cases

noOfDays...

```
1 from datetime import date
2
3 # Read input dates
4 d1 = input().split('-')
5 d2 = input().split('-')
6
7 # Convert to integers and create date objects
8 date1 = date(int(d1[0]), int(d1[1]), int(d1[2]))
9 date2 = date(int(d2[0]), int(d2[1]), int(d2[2]))
10
11 # Calculate difference
12 diff = date2 - date1
13
14 # Print number of days
15 print(diff.days)
```

Average time0.008 s8.00 ms

Maximum time0.010 s10.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 110 ms

Expected output

2014-07-02

2014-11-02

125

Actual output

2014-07-02

2014-11-02

125

Test case 27 ms

Terminal

Test cases

< Prev

Reset

Submit

Next >

1.1.2. Conditional Calculation Based on the Number of Digits

Write a Python program that accepts an integer n as input. Depending on the number of digits in n .

Constraints:
 $1 \leq n \leq 999$

Input Format:
• The input consists of a single integer n .

Output Format:
• If n is a single-digit number, print its square.
• If n is a two-digit number, print its square root (rounded to two decimal places).
• If n is a three-digit number, print its cube root (rounded to two decimal places).
• Else print "Invalid".

Sample Test Cases

condition...

```
1 #write your code here...
2 num = int(input())
3 if(0<num<10):
4     print(num*num)
5 elif(10<=num<100):
6     print(f"{num**(1/2):.2f}")
7 elif(100<=num<1000):
8     print(f"{num**(1/3):.2f}")
9 else:
10    print("Invalid")
```

Average time
0.004 s
4.14 ms

Maximum time
0.007 s
7.00 ms

4 out of 4 shown test case(s) passed
3 out of 3 hidden test case(s) passed

Test case 1 7 ms

Expected output
9
81

Actual output
9
81

Test case 2 4 ms

Test case 3 7 ms

Terminal

Test cases

< Prev

Reset

Submit

Next >

1.1.1. Calculate Momentum

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula:

$$p = m \times v$$

where:

m is the mass of the object (in kilograms)
 v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Sample Test Cases

```
1 #Type Content here...
2 m = float(input())
3 v = float(input())
4 print(f"{m * v:.2f}kgm/s")
```

Average time 0.013 s Maximum time 0.016 s
13.25 ms 16.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1	10 ms	Debug		
Expected output		Actual output		
5.0		5.0		
10.0		10.0		
50.00kgm/s		50.00kgm/s		
Test case 2	13 ms			

Terminal Test cases